MACHINE LEARNING TECHNICAL REPORT

Freight Rate Prediction Challenge

End-to-End Data Preprocessing, Feature Engineering, Modeling, and Inference Pipeline

1. Executive Summary

This report documents the design, implementation, and validation of an automated machine learning pipeline developed to predict freight load pricing (`posted_rate`) across commercial trucking lanes. Built upon historical load records (`data/train-test.csv`), the solution addresses real-world data anomalies, captures spatial and temporal pricing dynamics, and prevents data leakage through strict chronological splitting and training-bound parameter isolation.

The finalized pipeline produces two verified deliverables: a 12,000-row validation forecast (`validation_predictions.csv`) and a 31-day seasonal trajectory analysis for December 2025 (`data/december-chart-inputs.csv`) along the fixed Lexington to Fort Wayne corridor.

2. Problem Formulation & Validation Strategy

Freight spot rates fluctuate according to route distance, shipment weight, equipment type, and seasonal demand. Because future pricing depends on temporal market shifts, evaluating models via random shuffling risks optimistic data leakage. To mirror production forecasting conditions, the labeled development dataset was sorted chronologically by `date` and partitioned using a time-based cutoff:

- **Training Set (`date ≤ 2025-08-31`):** Used exclusively to learn imputation statistics, interquartile outlier bounds, skewness transformations, categorical encodings, and model weights.
- **Holdout Test Set (`date > 2025-08-31`):** Used to evaluate out-of-time generalization using Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and Coefficient of Determination (R^2).

3. Data Cleaning & Preprocessing

Exploratory data analysis identified two primary data quality issues in shipment records that required deterministic correction prior to modeling:

Pipeline Stage	Target Column(s)	Treatment & Leakage Prevention
Sign Correction	`weight`	Negative weight entries were converted to positive magnitudes via absolute value transformation (`.abs()`).
Missing Imputation	`weight`	Missing values were imputed using median statistics derived strictly from the training partition.

Pipeline Stage	Target Column(s)	Treatment & Leakage Prevention
Outlier Clamping	Numeric Features & `posted_rate`	Values were clipped to $[Q_1 - 1.5 \times IQR, Q_3 + 1.5 \times IQR]$ computed exclusively on `X_train` and `y_train`.
Skewness Normalization	Non-negative Numeric Columns	Variables exhibiting absolute skewness $\lvert sk \rvert \geq 1.0$ were stabilized using $log(1 + x)$ and inverted during inference via $exp(y) - 1$.

4. Feature Engineering & Categorical Encoding

To enrich the predictive signal available to tree-based regressors, domain-specific features were engineered and raw high-cardinality identifiers were transformed into numerical representations:

- **Temporal Decomposition:** Extracted `month` and `day` components from parsed timestamps to capture calendar seasonality and intra-month billing cycles.
- **Load Density Metric:** Engineered `weight_distance_ratio` (`weight / distance`) to quantify freight density per mile traveled.
- **Origin-Destination Route Interaction:** Combined `pickup` and `delivery` into a composite `route` identifier (`pickup_delivery`) to capture lane-specific supply-demand imbalances.
- **Multi-Strategy Encoding:** Applied normalized frequency encoding to individual `pickup` and `delivery` cities, out-of-fold cross-validated `TargetEncoder(cv=4)` to the composite `route` feature, and `OneHotEncoder` to `equipment` categories.
- **Feature Selection:** Dropped raw identifiers, timestamps, raw geospatial coordinates (`pickup_lat`, `pickup_lon`, `delivery_lat`, `delivery_lon`), and `market_index` after feature extraction to prevent collinearity and overfitting.

5. Model Architecture, Tuning & Codebase Structure

Gradient Boosted Decision Trees (`XGBRegressor`) and `RandomForestRegressor` were evaluated using `RandomizedSearchCV` (15 iterations optimizing `neg_root_mean_squared_error`). Optimal hyperparameters were serialized to `config/best_params.pkl`, and the trained estimator was persisted to `config/best_model.pkl`.

Module / File	Architectural Responsibility
`src/preprocess.py`	Cleans negative shipment weights and applies median imputation.
`src/feature_engineering.py`	Generates temporal, density, and route features; executes frequency, target, and one-hot encoding.
`modeling/tune.py`	Performs randomized hyperparameter search and exports optimal configurations to `config/best_params.pkl`.

Module / File	Architectural Responsibility
`main.py`	Executes end-to-end chronological splitting, IQR clamping, skewness adjustment, model training, evaluation (RMSE, MAE, R^2), and model serialization.
`infer.py`	Loads `config/best_model.pkl`, applies training-fitted transformations, and populates submission templates without retraining.

6. Verification & Final Deliverables

Running `infer.py` generated predictions for all 12,000 validation loads and all 31 daily intervals in December 2025. Both output files were verified against the official automated validation suite (`score.py`):

> **Automated Validation Summary:**
> • **Validation Set (`validation_predictions.csv`):** Confirmed exact schema (`load_id,predicted_rate`), 12,000 unique IDs (`TE-000001` to `TE-012000`), and strictly positive rate forecasts.
> • **December Trend Analysis (`data/december-chart-inputs.csv`):** Verified 31 daily predictions (Dec 1–31, 2025) for a fixed 360-mile Dry Van load (32,000 lb) from Lexington to Fort Wayne, generating `scorer_results/candidate_december.png`.